

Índice general

1	Introducción	2
2	Organización del proyecto	2
2.1	Arquitectura del sistema	3
3	Filosofía	3
4	Flujo general de construcción	3
5	Arquitectura	4
5.1	El Makefile principal	4
5.2	compila_figuras.sh	4
5.3	build.el	4
5.3.1	Ejecución selectiva de bloques Babel	5
5.4	Makefile.figuras	5
5.4.1	latexmk	6
5.4.2	pdftoppm	6
5.5	Responsabilidades de cada componente	6
6	Uso del sistema	6
6.1	Añadir una nueva lección	6
6.2	Compilación incremental	7
6.3	Limpieza del proyecto	7
7	Conclusiones	7
7.1	Ventajas del diseño	7

Generación automática de figuras PNG a partir de documentos Org

Marcos Bujosa Brun

17 de julio de 2026

1 Introducción

Este documento describe la infraestructura utilizada para generar automáticamente las figuras de las lecciones a partir de documentos **Org Mode**. El objetivo principal es separar claramente:

- el contenido docente (las figuras),
- la lógica de compilación,
- y el procedimiento de construcción.

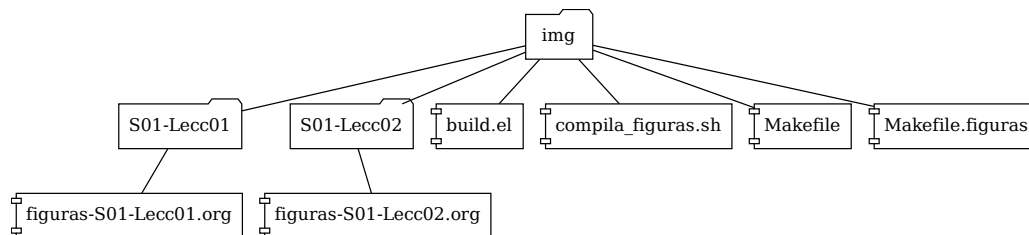
La filosofía seguida es la habitual en proyectos de software:

- cada componente tiene una única responsabilidad;
- la infraestructura se comparte entre todas las lecciones;
- añadir una nueva lección no requiere modificar el sistema de construcción.

El resultado es un sistema escalable en el que cada lección contiene únicamente la descripción de sus figuras.

2 Organización del proyecto

La estructura del directorio es la siguiente:



Cada subdirectorio contiene únicamente el documento **Org** correspondiente a una lección.

2.1 Arquitectura del sistema

El sistema está organizado en cuatro componentes claramente diferenciados, cada uno con una responsabilidad bien definida:

- `Makefile`: coordina la construcción completa del proyecto.
- `compila_figuras.sh`: procesa una lección concreta.
- `build.el`: procesa el documento Org mediante Org Babel.
- `Makefile.figuras`: compila los documentos $\text{\LaTeX}$ generados y obtiene las imágenes PNG.

Esta separación permite mantener cada componente sencillo e independiente, reduciendo el acoplamiento entre ellos y facilitando su mantenimiento.

3 Filosofía

El principio de diseño consiste en separar claramente la descripción de las figuras (documentos Org) de la infraestructura de construcción (scripts, Makefiles y código Elisp), de modo que cualquier lección pueda incorporarse al proyecto sin duplicar herramientas ni lógica de compilación.

Cada documento Org constituye una descripción declarativa de las figuras de una lección.

Los bloques Babel describen únicamente:

- el código $\text{\LaTeX}$ /TikZ necesario para producir cada figura;
- algunos pasos finales de posprocesado (por ejemplo, llamadas a `convert` de ImageMagick).

No contienen ninguna información sobre cómo compilar dichas figuras.

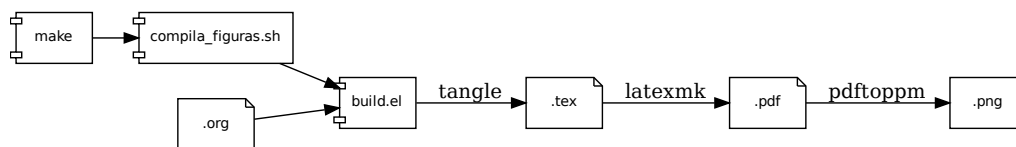
En particular, los documentos Org ya no incluyen:

- reglas `make`;
- llamadas a `latexmk`;
- llamadas a `make pngs`;
- infraestructura repetida en todas las lecciones.

Esta separación permite mantener una única copia de la lógica de construcción.

4 Flujo general de construcción

El procedimiento completo es



Cada una de estas etapas está controlada por un componente distinto.

5 Arquitectura

5.1 El Makefile principal

El Makefile situado en el directorio `img` constituye el punto de entrada del sistema.

Su misión consiste en

- recorrer todos los subdirectorios;
- localizar el documento Org correspondiente;
- decidir si es necesario reconstruir las figuras;
- invocar `compila_figuras.sh` únicamente cuando procede;
- proporcionar objetivos auxiliares como `clean` y `distclean`.

5.2 `compila_figuras.sh`

El script `compila_figuras.sh` constituye el trabajador encargado de procesar una única lección.

Recibe como argumento el documento Org correspondiente, invoca Emacs en modo batch utilizando `build.el` y, una vez generados los ficheros `.tex`, delega su compilación en `make` mediante `Makefile.figuras`.

5.3 `build.el`

El fichero `build.el` constituye el puente entre Org Babel y el sistema de construcción.

Su responsabilidad no es generar las imágenes finales, sino transformar el documento Org en los ficheros fuente necesarios para ello y ejecutar aquellas tareas de posprocesado que el autor haya marcado explícitamente mediante la cabecera `:build yes`.

Durante su ejecución realiza las siguientes operaciones:

1. carga Org y Org Babel;
2. habilita los lenguajes Babel necesarios;
3. abre el documento Org indicado;
4. crea el directorio `tex/` si todavía no existe;

5. ejecuta `org-babel-tangle` para generar los ficheros fuente;
6. ejecuta únicamente los bloques `bash` marcados con `:build yes`.

La idea es que el documento `Org` permanezca completamente independiente del mecanismo de construcción.

5.3.1 Ejecución selectiva de bloques Babel

Uno de los problemas encontrados durante el desarrollo fue que `(org-babel-execute-buffer)` ejecuta absolutamente todos los bloques del documento.

Eso produce diversos inconvenientes:

- intenta ejecutar bloques `latex` o `tex`;
- intenta interpretar resultados que no interesan;
- aparecen mensajes de error innecesarios.

Por este motivo se adoptó una estrategia distinta.

Los únicos bloques ejecutados automáticamente son aquellos que cumplen simultáneamente:

- son bloques `bash`;
- poseen el parámetro `build yes`

Por ejemplo,

```
#+begin_src bash :build yes
convert Figura.png -trim Figura.png
```

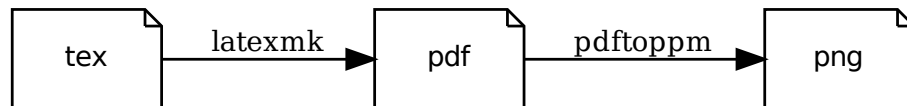
Esta decisión convierte el documento `Org` en una descripción declarativa del proceso de construcción.

5.4 Makefile.figuras

La compilación de las figuras no pertenece a `Org Babel`.

Corresponde a `make`.

Las reglas principales son



Para ello se utilizan dos herramientas especializadas.

5.4.1 latexmk

La generación del PDF se delega en `latexmk` en lugar de invocar directamente `pdflatex` porque `latexmk`

- ejecuta tantas compilaciones como sean necesarias;
- detecta automáticamente bibliografías y referencias;
- constituye la herramienta estándar para automatizar compilaciones L^AT_EX.

5.4.2 pdftoppm

La conversión de PDF a PNG se realiza mediante `pdftoppm` utilizando una resolución suficientemente elevada para obtener imágenes de buena calidad.

Cada fichero `.tex` genera automáticamente un fichero `.png`.

5.5 Responsabilidades de cada componente

El diseño final reparte las responsabilidades del siguiente modo.

Componente	Responsabilidad
documento Org	describir las figuras
Makefile	coordinar la construcción completa
compila_figuras.sh	procesar una lección
build.el	tanglar y ejecutar los bloques <code>:build yes</code>
Makefile.figuras	compilar <code>.tex</code> en <code>.pdf</code> y <code>.png</code>

Cada componente tiene una única responsabilidad.

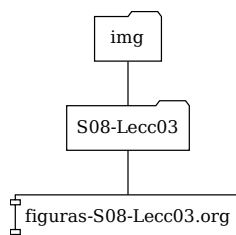
Esta separación facilita enormemente el mantenimiento.

6 Uso del sistema

6.1 Añadir una nueva lección

Incorporar una nueva lección únicamente requiere crear un nuevo subdirectorio que contenga el correspondiente documento Org.

Por ejemplo,



No es necesario copiar scripts ni modificar ningún fichero de configuración.

La nueva lección será detectada automáticamente por el Makefile, que localizará el nuevo subdirectorio y delegará su procesamiento en `compila_figuras.sh`.

6.2 Compilación incremental

El Makefile principal compara la fecha de modificación del documento Org de cada lección con las imágenes PNG generadas.

Únicamente cuando el documento Org es más reciente (o cuando las imágenes todavía no existen) se invoca `compila_figuras.sh`. De este modo, una ejecución repetida de `make` reconstruye únicamente aquellas lecciones que realmente han cambiado, reduciendo considerablemente el tiempo de construcción del proyecto.

6.3 Limpieza del proyecto

El Makefile principal proporciona dos objetivos auxiliares:

- `make clean`: elimina los directorios `tex/` generados durante la compilación.
- `make distclean`: además de eliminar los directorios `tex/`, borra todos los ficheros generados automáticamente, conservando únicamente los documentos fuente `.org`.

Estos objetivos permiten recuperar fácilmente un árbol de directorios limpio sin perder ninguno de los documentos originales.

7 Conclusiones

7.1 Ventajas del diseño

La arquitectura presenta varias ventajas importantes.

- No existe código duplicado.
- Toda la lógica de construcción está centralizada.
- Cada documento Org contiene únicamente información docente.
- El sistema puede ampliarse sin modificar la infraestructura.

- Resulta sencillo cambiar herramientas de compilación (por ejemplo, sustituir `pdflatex` por `lualatex` o modificar la resolución de las imágenes) editando un único fichero.
- La infraestructura queda claramente separada del contenido.
- La compilación puede realizarse de forma incremental: únicamente se reconstruyen las lecciones que lo requieren.

En conjunto, el proyecto adopta una filosofía muy próxima a la programación literaria: los documentos Org describen el conocimiento (las figuras y su significado), mientras que la infraestructura de construcción permanece desacoplada y reutilizable para todas las lecciones.